

Aulão de Banco de Dados

ENADE — a partir do Simulado 2026-1

Tópicos Especiais II · ENADE · 2026/2

Prof. M.Sc. Howard Cruz Roatti

O diagnóstico, em uma frase

Banco de Dados caiu **só em SI e ADS** (3 itens cada) — e os dois estão em velocidades muito diferentes:

SI — 83,3%

Conceito de pé. Faltam **pontos finos**.

ADS — 47,1%

Um **buraco** claro, concentrado.

💡 O eixo do aulão sai limpo dos dados: **o problema não é junção nem subconsulta — é agregação e agrupamento** (GROUP BY , SUM , AVG).

Abertura — votem no ar 🙋

Banco de eleição: Candidato(numero, nome), Partido(numero, nome, sigla), Votacao(partido, votos, estado). Queremos o total de votos de cada partido/candidato. Todas partem da mesma base — muda só o destacado:

```
SELECT c.nome, p.nome, «AGREGADO» FROM Partido p, Candidato c, Votacao v
WHERE c.numero = p.numero AND v.partido = c.numero «AGRUPAMENTO»;
```

	«AGREGADO»	«AGRUPAMENTO»
A	COUNT(v.votos)	GROUP BY c.nome, p.nome
B	SUM(v.votos)	GROUP BY c.nome, p.nome
C	SUM(v.votos)	(sem GROUP BY)
D	SUM(v.votos)	GROUP BY c.nome, p.nome, v.votos

🙋 A, B, C ou D? Guardem a resposta. Voltamos a ela em 3 minutos.

O contraste que orienta a aula

SI · Q14 — subconsulta IN tripla

```
SELECT ... WHERE idMoto IN (SELECT ... IN  
(SELECT ...))
```

100% de acerto

Três níveis aninhados **não derrubaram ninguém**.

💡 Por que o "mais difícil" foi o mais fácil? **Subconsulta é procedural** — resolve de dentro para fora, como código. **Agregação é declarativa** — exige pensar em **conjuntos**. É aí que a intuição de quem programa falha.

ADS · Q16 — SUM com GROUP BY

A consulta de agregação da abertura.

6% de acerto

1 acerto em 17. 12 na mesma errada.

Roteiro de hoje

#	Item	Assunto	Tempo
1	ADS Q16	SUM com GROUP BY	15 min
2	ADS Q17	AVG , GROUP BY e ORDER BY	12 min
3	SI Q15	JOIN com apelidos e coluna ambígua	8 min
4	ADS Q19	Modelo ER e chave candidata	7 min
5	SI Q13	RIGHT × LEFT × INNER JOIN	6 min
6	ADS Q23	Diagrama de classes e multiplicidade	3 min

💡 Núcleo em **agregação** (segura ADS); junção e modelagem entram como **revisão rápida** (para SI).

1 · ADS Q16

SUM com **GROUP BY** — vamos rodar as duas

Q16 — rode as duas à mão

Seis linhas de `Votacao` e a saída de **cada** consulta:

Dados (`Votacao`)

partido	votos
Alfa	100
Alfa	50
Alfa	30
Beta	200
Beta	20

B — `GROUP BY c.nome, p.nome` ✓

candidato	partido	SUM
Ana	Alfa	180
Bruno	Beta	220

D — `... , v.votos` ✗

candidato	partido	SUM
Ana	Alfa	100
Ana	Alfa	50
...	...	...

⚠ Agrupar **também** por `v.votos` cria **um grupo para cada valor de voto** → a soma vira o próprio valor.
A totalização **some**, mas o banco aceita e devolve linhas.

Q16 — a regra (só agora) + gabarito B

💡 **Agrupa-se pelo que NÃO está agregado.** As colunas do `SELECT` que não entram numa função (`SUM`, `AVG` ...) vão **todas**, e só elas, no `GROUP BY`.

- **B** ✓ — `SUM(v.votos)` totaliza; `GROUP BY c.nome, p.nome` dá o total por partido/candidato.
- **D** ✗ (12 alunos) — agrupar por `v.votos` anula a soma (o erro campeão).
- **A** ✗ — usa `COUNT` (conta linhas, não soma votos).
- **C** ✗ — `SUM` **sem** `GROUP BY` com colunas não agregadas → consulta inválida.

📌 O que o erro revela: quem marca D acha que `GROUP BY` "lista as colunas do `SELECT`". Enquanto essa ideia estiver de pé, **toda** totalização sai errada.

2 · ADS Q17

AVG, GROUP BY e ORDER BY

Q17 — primeiro, em português

Banco bancário: `CLIENTE`, `CONTA`, `HISTORICO_MOVIMENTACAO`. Queremos **o nome de cada cliente e o valor médio movimentado por ele, do maior para o menor**.

💡 Antes de qualquer SQL: qual a diferença entre "**o valor movimentado**" e "**a média por cliente**"? Quem não enuncia isso não escreve o `GROUP BY` — por mais sintaxe que decore.

Sem agregar (erro B, 4 alunos)

cliente	valor
Bruno	50
Ana	100
Bruno	150
Ana	200

AVG ... GROUP BY cliente ✓

cliente	média
Ana	150
Bruno	100

Q17 — gabarito A

```
SELECT CL.NOME, AVG(HM.VAL_MOVIMENTADO)
FROM CLIENTE CL
  JOIN CONTA CO ON CL.COD_CLIENTE = CO.COD_CLIENTE
  JOIN HISTORICO_MOVIMENTACAO HM ON CO.NUM_CONTA = HM.NUM_CONTA
GROUP BY CL.COD_CLIENTE, CL.NOME
ORDER BY AVG(HM.VAL_MOVIMENTADO) DESC;  -- ordena pela própria agregação
```

- **A**  — AVG + GROUP BY por cliente + ORDER BY AVG(...) DESC.
- **B**  — lista os valores **individuais** (sem AVG, sem agrupar).
- **C**  — AVG **sem** GROUP BY.
- **D**  — junções erradas (colunas trocadas).

💡 Sete dos dez erros de ADS foram de agregação — **não** de junção. E o item ainda cobra o passo seguinte: **ordenar pela função de agregação**.

3 · SI Q15

JOIN, apelidos e coluna ambígua

Q15 — o conceito está de pé, faltam os finos

Peca(CodPeca, NomePeca, ...) e Embarque(CodPeca, CodFornecedor, QuantidadeEmbarque). Queremos CodPeca e NomePeca das peças com QuantidadeEmbarque > 100.

```
-- A (correta)
SELECT P.CodPeca, P.NomePeca
FROM Peca P JOIN Embarque E ON P.CodPeca = E.CodPeca
WHERE E.QuantidadeEmbarque > 100;
```

🧠 Pergunta que resolve as duas pegadinhas: por que WHERE QuantidadeEmbarque > 100 sem qualificar às vezes funciona e às vezes não? (Obriga a pensar no **esquema**, não na consulta.)

- **B** ✗ — FROM Peca Embarque lê Embarque como **apelido** de Peca.
- **C** ✗ — CodPeca **sem qualificar** é **ambíguo** (existe nas duas tabelas).
- **D** ✗ — Embarque só existe na subconsulta; referenciá-la no SELECT externo é **fora de escopo**.

4 · ADS Q19

Modelo ER e chave candidata — a ponte para o SQL

Q19 — leia a cardinalidade em voz alta

pessoa **possui** cavalo : cada pessoa possui **exatamente um** cavalo $(1,1)$; cada cavalo pertence a **no máximo uma** pessoa $(0,1)$.

💡 Traduza cada número: o **primeiro é o mínimo**, o **segundo é o máximo**. $(0,1)$ = "de zero a um" → **opcional**.

- **C** ✓ — o `rg` identifica unicamente cada pessoa → pode ser **chave candidata**.
- **D** ✗ (3 alunos) — leram o **mínimo 0** de $(0,1)$ como **obrigatório**; confundem "no máximo uma" com "exatamente uma".
- **A** ✗ — $(1,1)$ limita a **um** cavalo por pessoa.
- **B** ✗ — `raca` / `eBento` são descritivos; a PK de `cavalo` é `codigo` .

✦ O que muda no esquema quando o mínimo é **0**? É onde entra a **chave estrangeira que aceita nulo** — modelagem virando SQL.

5 · SI Q13

RIGHT × **LEFT** × **INNER** — 30 segundos por junção

Q13 — junção é operação de conjunto

Queremos **todas as linhas de Tabela2** e as correspondentes de Tabela1.

- **INNER** — só a **interseção**
- **LEFT** — tudo da **esquerda** + interseção
- **RIGHT** — tudo da **direita** + interseção

```
-- A (correta)
SELECT *
FROM Tabela1 t1
RIGHT JOIN Tabela2 t2
  ON t1.id = t2.fk;
```

- **C** ✗ (2 alunos) — **LEFT** preserva **Tabela1**, não Tabela2 (lado trocado).
- **D** ✗ — **INNER** devolve só a interseção.

💡 Bônus (quase ninguém conhece): `RIGHT JOIN ... WHERE t1.id IS NULL` é o idioma para "**o que existe de um lado e não do outro**".

6 · ADS Q23 — multiplicidade UML (não gastar aula)

Estudante 1..* Matrícula; cada Matrícula liga-se a um Curso (via Turma); Turma tem Horário e Professor.

Gabarito A — 94,1% de acerto (16/17).

💡 Use como **argumento com a própria turma**: vocês **modelam bem** (UML, multiplicidade, ER). O que falta é **traduzir o modelo em consulta** — que é exatamente a agregação de hoje.

🔗 Aprofundar modelagem → decks de BD: **Modelagem Conceitual (ER)** e **Lógica (→ tabelas)**, com os exercícios integradores.

Fecho — a regra e o que estudar por conta

💡 **A frase do aulão:** em agregação, **agrupa-se pelo que não está agregado**; funções (SUM / AVG / COUNT) resumem cada grupo; HAVING filtra grupos; ORDER BY pode ordenar pela própria agregação.

O simulado NÃO mediu (e o ENADE cobra) — revise nos decks de BD:

- **Normalização** (o mais recorrente!) → deck *Normalização*
- **Transações e ACID** → deck *Transações*
- **Concorrência / isolamento** → deck *Concorrência*
- **Índices e desempenho** → deck *Views, SQL e Indexação*
- **Álgebra relacional** → deck *Álgebra Relacional*
- **NoSQL e CAP** → deck *NoSQL*

Bons estudos!

howard.cruz@faesa.br

≡ Índice

Tópicos Especiais II

Aula 2 · Banco de Dados

revisão completa